Product ontology for a team of agents: memo

A product ontology is a set of files in a `product/` folder next to the code. It records what the product consists of, how its parts are connected, and which decisions and requirements apply, with references to the code. The ontology lives in the repository, and an agent on any model can read it.

What it gives

- The agent sees links that are not in the text of the code: where a screen leads, which paths create a document, what the product owner decided.
- The agent takes only the record it needs and does not spend tokens reading the whole ontology.
- A script, with no model involved, checks whether an agent's change meets the requirements in the ontology.

Where no benefit shows

- Code the model already knows: popular open-source systems.
- Questions whose answer a search through the code finds.
- The cheapest model: with the ontology it finds less and spends more.
- The ontology of a large system read in full: the agent spends more tokens than without it.

Two layers

- **Requirements and decisions** are written by people from day one, straight away in a checkable form. The model cannot recover the product owner's intent from the code.
- **The structure** is built by a model once the data schema stops changing every day. A check script finds records that no longer match the code.

What to record

Requirements and decisions are recorded at any level of detail.

- **Map:** the parts of the product and where they live in the code.
- **Documents and links:** entities, conversions between screens and documents, every path that creates a document. This is the main level; change checks work on it.
- **Fields:** where a field is filled from, where it is carried to, where it is printed and shown in reports. This level costs more to build and to read. On an open-source system it did not raise answer quality; on a closed one it has not been tested yet.
- **Functions and calls:** the code graph. Code analysis tools build it; it is not part of the ontology.

How to give it to an agent

- **Developer agent:** the index and the records it needs, by search.
- **Product owner and analyst agents:** decisions and requirements in full, everything else on request.

A requirement an agent can follow

- **source of the value:** the document the value comes from; in review it shows that the agent put in something of its own;
- **exceptions** are listed explicitly; an empty list means there are none;
- **where it applies:** every path that creates the document;
- **open questions:** while there are any, a change to this field does not pass the check;
- **type:** a product violation goes back to the product owner, a technical one is fixed by the author.

People approve the decisions an agent made on its own in review, like code.

Checks

- `check.py` checks the ontology against the code. It runs in CI on every change.
- `check_change.py` checks a change against the requirements: creation paths, stubs, weakening of the rule. The script sees only what is recorded in the ontology. A reviewer is responsible for the completeness of paths and for where the value comes from.

Agent team

- The model is chosen by the difficulty of the work: a strong model does analysis and the final review, a mid-range one writes code and texts, a cheap one goes through a checklist.
- At the end, a lead on a strong model reads the tester's report and fixes the hard parts.
- Models are compared by the cost of the whole task: a model that is expensive per token may spend fewer tokens on the task.

Where to start

With one part of the system where a mistake costs most: documents, money, access rights, mandatory fields. For an existing product, the part that is being changed now.

Meetup "AI in software development", 20 October 2026. Nicolay Amiel Trzhascal, FabricaONE.AI. Scripts and template: `02_ontology_kit`; models by role: `03_agent_team`.